

Giáo viên hướng dẫn	Thân Thế Tùng		Điểm
Họ và tên	Vũ Đại Dương	23520359	

CE119-Lab04

1. Thực hành với thủ tục

¶

```

.data
prompt: .ascii "Enter one number: "

.text
main:   jal getInt
        move $s0, $v0
        j exit

getInt: li $v0, 4
        la $a0, prompt
        syscall

        li $v0, 5
        syscall
        jr $ra

exit:

```

Hình 1: nhập từ cửa sổ I/O một số nguyên vào thanh ghi \$s0

```

. . .
        move $a0, $s0
        move $a1, $s1
        move $a2, $s2
        move $a3, $s3
        jal proc_example

        move $a0, $v0
        li $v0, 1
        syscall

. . .
proc_example:
        addi $sp, $sp, -4
        sw $s0, 0($sp)

        add $t0, $a0, $a1
        add $t1, $a2, $a3
        sub $s0, $t0, $t1

        move $v0, $s0

        lw $s0, 0($sp)
        addi $sp, $sp, 4

        jr $ra

```

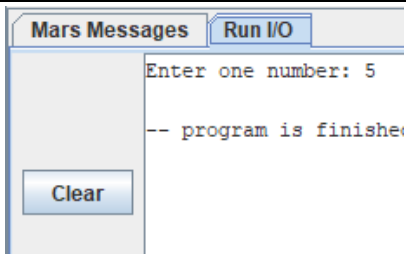
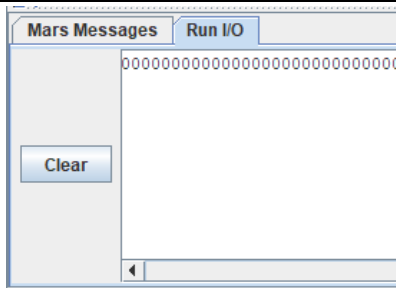
Hình 2: Phép toán (a + b) - (c + d)

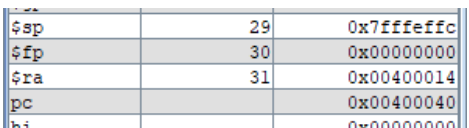
- Chạy từng bước và theo dõi sự thay đổi của thanh ghi PC, \$ra, \$sp, \$fp trong 2 hình trên.

	Hình 1	Hình 2
PC	0x00400000 -> 0x0040000c -> 0x00400010 -> 0x00400014 -> 0x00400018 -> 0x0040001c -> 0x00400020 -> 0x00400024 -> 0x00400004 -> 0x00400008 -> 0x00400028	0x00400000 -> 0x00400004 -> 0x00400008 -> 0x0040000c -> 0x00400010 -> 0x00400020 -> 0x00400024 -> 0x00400028 -> 0x0040002c -> 0x00400030 -> 0x00400034 -> 0x00400038 -> 0x0040003c -> 0x00400040 ->

	- 0x00400000 -> 0x0040000c (thay đổi sau khi chạy lệnh jal => nhảy đến địa chỉ của getInt) - Sau đó pc = pc + 4 cho đến lệnh jr \$ra - 0x00400024 -> 0x00400004 (thay đổi sau khi chạy lệnh jr => nhảy về địa chỉ \$ra) - Sau đó pc = pc + 4 cho đến lệnh j exit - 0x00400008 -> 0x00400028 (thay đổi sau khi chạy lệnh j => chạy đến địa chỉ của exit)	0x00400014 -> 0x00400018 -> 0x0040001c -> 0x00400020 -> ... (lặp lại từ 0x00400020) - pc = pc + 4 cho đến lệnh jal proc_example - 0x00400010 -> 0x00400020 (thay đổi sau khi chạy xong lệnh jal => nhảy đến địa chỉ của proc_example) - Sau đó pc = pc + 4 cho đến lệnh jr \$ra - 0x00400040 -> 0x00400014 (thay đổi sau khi chạy xong lệnh jr => nhảy về địa chỉ mà thanh ghi \$ra đang lưu) - Sau đó pc = pc + 4 cho đến lệnh jr \$ra => tạo thành vòng lặp vô tận																				
\$ra	0x00000000 -> 0x00400004 (thay đổi sau khi chạy lệnh jal)	0x00000000 -> 0x00400014 (thay đổi sau khi chạy lệnh jal)																				
\$sp	0x7ffefffc (không thay đổi)	0x7ffefffc -> 0x7ffefff8 -> 0x7ffefffc																				
\$fp	0x0000000 (không thay đổi)	0x00000000 (không thay đổi)																				
Địa chỉ các hàm:	<table><tr><th>Label</th><th>Address ▼</th></tr><tr><td colspan="2">Hinh1.asm</td></tr><tr><td>prompt</td><td>0x10010000</td></tr><tr><td>exit</td><td>0x00400028</td></tr><tr><td>getInt</td><td>0x0040000c</td></tr><tr><td>main</td><td>0x00400000</td></tr></table>	Label	Address ▼	Hinh1.asm		prompt	0x10010000	exit	0x00400028	getInt	0x0040000c	main	0x00400000	<table><tr><th>Label</th><th>Address ▼</th></tr><tr><td colspan="2">Hinh2.asm</td></tr><tr><td>proc_example</td><td>0x00400020</td></tr><tr><td>main</td><td>0x00400000</td></tr></table>	Label	Address ▼	Hinh2.asm		proc_example	0x00400020	main	0x00400000
Label	Address ▼																					
Hinh1.asm																						
prompt	0x10010000																					
exit	0x00400028																					
getInt	0x0040000c																					
main	0x00400000																					
Label	Address ▼																					
Hinh2.asm																						
proc_example	0x00400020																					
main	0x00400000																					

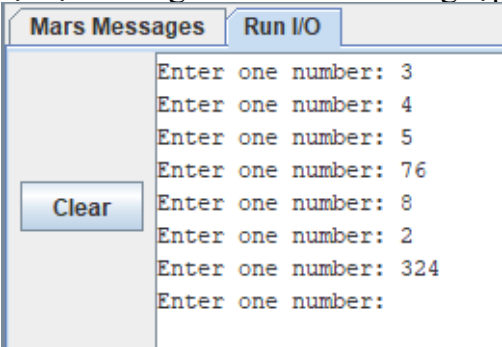
- Chạy toàn chương trình một lần để xem kết quả

	Hình 1	Hình 2																								
Chạy I/O		 - In ra số 0 vô tận.																								
Bảng thanh ghi (\$sp, \$fp, \$ra, pc)	<table border="1"> <tbody> <tr> <td>\$sp</td> <td>29</td> <td>0x7ffefffc</td> </tr> <tr> <td>\$fp</td> <td>30</td> <td>0x00000000</td> </tr> <tr> <td>\$ra</td> <td>31</td> <td>0x00400004</td> </tr> <tr> <td>pc</td> <td></td> <td>0x00400028</td> </tr> </tbody> </table>	\$sp	29	0x7ffefffc	\$fp	30	0x00000000	\$ra	31	0x00400004	pc		0x00400028	<table border="1"> <tbody> <tr> <td>\$sp</td> <td>29</td> <td>0x7ffefffc</td> </tr> <tr> <td>\$fp</td> <td>30</td> <td>0x00000000</td> </tr> <tr> <td>\$ra</td> <td>31</td> <td>0x00400014</td> </tr> <tr> <td>pc</td> <td></td> <td>0x00400040</td> </tr> </tbody> </table>	\$sp	29	0x7ffefffc	\$fp	30	0x00000000	\$ra	31	0x00400014	pc		0x00400040
\$sp	29	0x7ffefffc																								
\$fp	30	0x00000000																								
\$ra	31	0x00400004																								
pc		0x00400028																								
\$sp	29	0x7ffefffc																								
\$fp	30	0x00000000																								
\$ra	31	0x00400014																								
pc		0x00400040																								

		 - Vì là vòng lặp nên chương trình luôn chạy -> thanh ghi pc sẽ luôn thay đổi không ngừng.
--	--	--

- Với code trong Hình 1, nếu bỏ dòng code “j exit”, việc gì sẽ xảy ra?

+ Với code trong Hình 1, nếu bỏ dòng code “j exit”, chương trình sau khi chạy xong hết lệnh trong hàm main, sẽ chạy tiếp phần getInt, sau đó sẽ đc trả về địa chỉ \$ra và thực hiện lại lệnh trong hàm main => vòng lặp vô tận.



- Viết lại code trong Hình 1, thêm vào thủ tục tên showInt để in ra cửa sổ I/O giá trị của số int nhập vào cộng thêm 1.

+ Sau khi chạy xong lệnh **move \$s0, \$v0**, thay vì j exit, ta sẽ j showInt để thực hiện yêu cầu, sau đó exit (Nếu showInt ngay trước hàm exit thì không cần j exit, còn không, sau khi thực hiện yêu cầu thì sử dụng lệnh j exit để kết thúc)

showInt ngay trước exit	showInt ở các vị trí khác
-------------------------	---------------------------

<pre> 1 .data 2 prompt: .ascii "Enter one number: " 3 4 .text 5 main: jal getInt 6 move \$s0, \$v0 7 j showInt 8 9 getInt: li \$v0, 4 10 la \$a0, prompt 11 syscall 12 13 li \$v0, 5 14 syscall 15 jr \$ra 16 17 showInt: 18 addi \$s0, \$s0, 1 19 li \$v0, 1 20 move \$a0, \$s0 21 syscall 22 23 exit: </pre>	<pre> 1 .data 2 prompt: .ascii "Enter one number: " 3 4 .text 5 main: jal getInt 6 move \$s0, \$v0 7 j showInt 8 9 showInt: 10 addi \$s0, \$s0, 1 11 li \$v0, 1 12 move \$a0, \$s0 13 syscall 14 j exit 15 16 getInt: li \$v0, 4 17 la \$a0, prompt 18 syscall 19 20 li \$v0, 5 21 syscall 22 jr \$ra 23 24 exit: </pre>
Mars Messages Run I/O Enter one number: 5 6 -- program is finished r Clear	

- Viết lại code trong Hình 2, lúc này chương trình chính cần tính giá trị của cả hai biểu thức: $(a + b) - (c + d)$ và $(a - b) + (c - d)$, hàm `proc_example` có hai giá trị trả về và trong thân hàm sử dụng hai biến cục bộ `$s0` và `$s1` (`$s1` lưu kết quả của $(a - b) + (c - d)$)

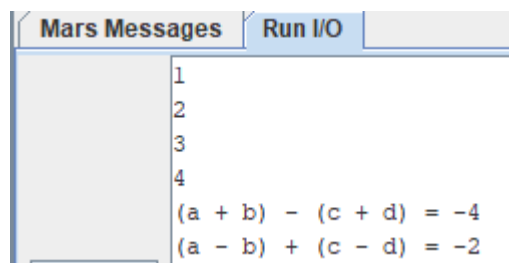
<pre> .data sumofex: .ascii "(a - b) + (c - d) = " exofsum: .ascii "(a + b) - (c + d) = " enter: .ascii "\n" .text main: li \$v0, 5 syscall move \$s0, \$v0 li \$v0, 5 syscall move \$s1, \$v0 </pre>	- Nhập a và lưu giá trị vào thanh ghi <code>\$s0</code> - Nhập b và lưu giá trị vào thanh ghi <code>\$s1</code>
---	--

li \$v0, 5 syscall move \$s2, \$v0	- Nhập c và lưu giá trị vào thanh ghi \$s2
li \$v0, 5 syscall move \$s3, \$v0	- Nhập d và lưu giá trị vào thanh ghi \$s3
move \$a0, \$s0 move \$a1, \$s1 move \$a2, \$s2 move \$a3, \$s3 jal proc_example	- Lưu giá trị a, b, c, d lần lượt vào thanh ghi \$a0, \$a1, \$a2, \$a3 - Thực hiện hàm proc_example, sau khi thực hiện xong hàm thì trả về vị trí pc = pc + 4 (lệnh tiếp theo sau jal)
move \$t0, \$v0	- Vì \$v0 còn được dùng để syscall, nên lưu giá trị \$v0 vào thanh ghi \$t0
li \$v0, 4 la \$a0, exofsum syscall	- In: $(a + b) - (c + d) =$
move \$a0, \$t0 li \$v0, 1 syscall	- In giá trị thanh ghi \$t0
li \$v0, 4 la \$a0, enter syscall	- Xuống dòng
li \$v0, 4 la \$a0, sumofex syscall	- In: $(a - b) + (c - d) =$
li \$v0, 1 move \$a0, \$v1 syscall	- In giá trị thanh ghi \$v1
j exit	- Kết thúc chương trình
proc_example:	- Thực hiện phép tính $(a + b) - (c + d)$ và

<pre> add \$t0, \$a0, \$a1 add \$t1, \$a2, \$a3 sub \$s0, \$t0, \$t1 sub \$t2, \$a0, \$a1 sub \$t3, \$a2, \$a3 add \$s1, \$t2, \$t3 move \$v0, \$s0 move \$v1, \$s1 jr \$ra exit: </pre>	lưu giá trị vào thanh ghi \$s0 - Thực hiện phép tính $(a - b) + (c - d)$ và lưu giá trị vào thanh ghi \$s1 - Lưu giá trị \$s0 vào thanh ghi \$v0 - Lưu giá trị \$s1 vào thanh ghi \$v1 - Trả về địa chỉ \$ra đang lưu trữ để tiếp tục chạy lệnh
--	---

Chạy mô phỏng:

a = 1, b = 2, c = 3, d = 4



- Viết lại code trong Hình 2, lúc này chương trình chính cần tính giá trị của cả hai biểu thức: $(a + b) - (c + d)$, $(e - f)$ hàm proc_example có 6 input và hai giá trị trả về và trong thân hàm sử dụng hai biến cục bộ \$s0 và \$s1 (\$s1 lưu kết quả của e - f)

<pre> .data ef: .asciiz "e - f = " exofsum: .asciiz "(a + b) - (c + d) = " enter: .asciiz "\n" .text main: li \$v0, 5 syscall move \$s0, \$v0 li \$v0, 5 syscall move \$s1, \$v0 </pre>	- Nhập a và lưu giá trị vào thanh ghi \$s0 - Nhập b và lưu giá trị vào thanh ghi \$s1
---	---

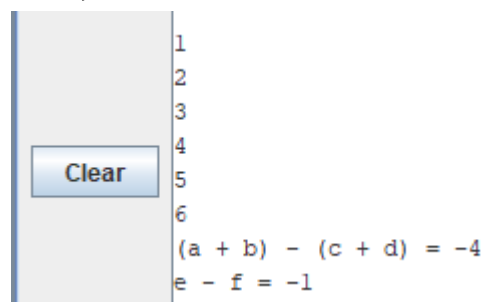
li \$v0, 5 syscall move \$s2, \$v0	- Nhập c và lưu giá trị vào thanh ghi \$s2
li \$v0, 5 syscall move \$s3, \$v0	- Nhập d và lưu giá trị vào thanh ghi \$s3
li \$v0, 5 syscall subu \$sp, \$sp, 4 sw \$v0, 0(\$sp)	- Nhập e và lưu giá trị vào phần tử -4 của stack
li \$v0, 5 syscall subu \$sp, \$sp, 4 sw \$v0, 0(\$sp)	- Nhập f và lưu giá trị vào phần tử -8 của stack
move \$a0, \$s0 move \$a1, \$s1 move \$a2, \$s2 move \$a3, \$s3 jal proc_example	- Lưu giá trị a, b, c, d lần lượt vào thanh ghi \$a0, \$a1, \$a2, \$a3
move \$t0, \$v0	- Thực hiện hàm proc_example, sau khi thực hiện xong thì trả về địa chỉ là lệnh sau jal
li \$v0, 4 la \$a0, exofsum syscall	- Vì \$v0 còn dùng để syscall, nên lưu giá trị \$v0 vào \$t0
move \$a0, \$t0 li \$v0, 1 syscall	- In: $(a + b) - (c + d) =$
li \$v0, 4 la \$a0, enter syscall	- In giá trị \$t0
li \$v0, 4 la \$a0, ef syscall	- Xuống dòng
li \$v0, 1	- In: $e - f$
	- In giá trị \$v1

<pre> move \$a0, \$v1 syscall j exit proc_example: lw \$t2, 4(\$sp) lw \$t3, 0(\$sp) add \$sp, \$sp, 8 add \$t0, \$a0, \$a1 add \$t1, \$a2, \$a3 sub \$s0, \$t0, \$t1 sub \$s1, \$t2, \$t3 move \$v0, \$s0 move \$v1, \$s1 jr \$ra exit: </pre>	- Kết thúc - Lấy giá trị e - Lấy giá trị f - Trả thanh ghi \$sp về giá trị trước khi lưu - Thực hiện phép tính $(a + b) - (c + d)$ và lưu giá trị vào thanh ghi \$s0 - Thực hiện phép tính $e - f$ và lưu giá trị vào thanh ghi \$s1 - Lưu giá trị \$s0 vào thanh ghi \$v0 - Lưu giá trị \$s1 vào thanh ghi \$v1 - Trả về địa chỉ \$ra đang lưu trữ để tiếp tục chạy lệnh
--	---

Chạy mô phỏng:

a = 1, b = 2, c = 3, d = 4

e = 5, f = 6



2. ¶Thực hành với đệ quy

Khi thân của một thủ tục gọi một thủ tục khác, giá trị của thanh ghi \$ra ngay lập tức bị ghi đè, xóa mất giá trị cũ, dẫn tới sau khi thủ tục này thực hiện xong sẽ không có địa chỉ để quay về. Vì vậy, nếu trong trường hợp lồng thủ tục hay đệ quy, thanh ghi \$ra cũng phải được lưu lại cùng các thanh ghi \$s.

Viết code MIPS, chạy kiểm thử trên MARS và giải thích ý nghĩa rõ ràng cho chương trình tính giai thừa được viết bằng code C như Hình 3.


```

main ()
{
    printf ("The factorial of 10 is %d\n", fact (10));
}

int fact (int n)
{
    if (n < 1)
        return (1);
    else
        return (n * fact (n - 1));
}

```

Hình 3: Tính giai thừa

.data prompt: .asciiz "Factorial of 10 is: " .text .globl main main: li \$a0, 10 jal factorial move \$s0, \$v0 li \$v0, 4 la \$a0, prompt syscall	- Lưu \$a0 = 10 - Gọi hàm factorial - Lưu giá trị \$v0 vào \$s0 (vì \$v0 còn dùng để syscall) - In: Factorial of 10 is:
--	---

li \$v0, 1	- In kết quả fact(10)
move \$a0, \$s0	
syscall	
li \$v0, 10	- Kết thúc chương trình
syscall	
factorial:	
blt \$a0, 1, base_case	- ($n < 1$) => đến hàm base_case
addi \$sp, \$sp, -8	
sw \$ra, 4(\$sp)	- Lưu giá trị \$ra vào stack vì sau khi gọi đệ quy thì giá trị \$ra sẽ thay đổi
sw \$a0, 0(\$sp)	- Lưu giá trị n vào stack vì sau khi gọi đệ quy thì giá trị n sẽ thay đổi
addi \$a0, \$a0, -1	- Giảm giá trị n đi 1
jal factorial	- Gọi đệ quy factorial(n-1) cho đến khi $n < 1$
	- Sau khi đệ quy thì khôi phục lại từng giá trị n và \$ra trong stack
lw \$a0, 0(\$sp)	- Lấy giá trị của n
lw \$ra, 4(\$sp)	- Lấy giá trị của \$ra
addi \$sp, \$sp, 8	- Tăng \$sp để tiếp tục lấy giá trị của các phần tử tiếp theo
mul \$v0, \$v0, \$a0	- $\$v0 = n * \text{factorial}(n-1)$
jr \$ra	- Quay lại gọi hàm

base_case: li \$v0, 1 jr \$ra	- \$v0 = 1 để thực hiện tính toán tại hàm factorial - Quay lại hàm gọi
---	---

Chạy mô phỏng:

```
Factorial of 10 is: 3628800
```

3. Bài tập

- a. Tiếp tục nội dung 2. Vẽ lại hình ảnh của các stack trong trường hợp tính 5! và 10! - 5!

	Địa chỉ cao
Địa chỉ \$ra (1)	
Giá trị \$a0	
Địa chỉ \$ra (2)	
Giá trị \$a0 - 1	
Địa chỉ \$ra (2)	
Giá trị \$a0 - 2	
Địa chỉ \$ra (2)	
Giá trị \$a0 - 3	
Địa chỉ \$ra (2)	
Giá trị \$a0 - 4	
Địa chỉ \$ra (2)	
Giá trị \$a0 - 5	← \$sp
	Địa chỉ thấp

- 10!

	Địa chỉ cao
Địa chỉ \$ra (1)	
Giá trị \$a0	
Địa chỉ \$ra (2)	
Giá trị \$a0 - 1	
Địa chỉ \$ra (2)	

Giá trị \$a0 – 2
Địa chỉ \$ra (2)
Giá trị \$a0 - 3
Địa chỉ \$ra (2)
Giá trị \$a0 – 4
Địa chỉ \$ra (2)
Giá trị \$a0 – 5
Địa chỉ \$ra (2)
Giá trị \$a0 – 6
Địa chỉ \$ra (2)
Giá trị \$a0 – 7
Địa chỉ \$ra (2)
Giá trị \$a0 – 8
Địa chỉ \$ra (2)
Giá trị \$a0 - 9
Địa chỉ \$ra (2)
Giá trị \$a0 - 10

Địa chỉ thấp

b. Viết chương trình nhập vào n và xuất ra chuỗi Fibonacci tương ứng

<pre> .text .globl main main: li \$v0, 4 la \$a0, prompt1 syscall li \$v0, 5 syscall move \$a0, \$v0 jal fibonacci move \$t0, \$v0 move \$t1, \$a0 li \$v0, 4 la \$a0, prompt2 syscall </pre>	- In prompt1 - Nhập n - Lưu n vào \$a0 - Thực hiện hàm fibonacci - Lưu giá trị fibo vào \$t0 - Lưu vị trí lấy giá trị vào \$t1 - In prompt2
---	---

<pre> li \$v0, 1 move \$a0, \$t1 syscall li \$v0, 4 la \$a0, prompt3 syscall li \$v0, 1 move \$a0, \$t0 syscall li \$v0, 10 syscall .data prompt1: .asciiz "Enter a value: " prompt2: .asciiz "Fibonacci (" prompt3: .asciiz ") = " .text .globl fibonacci fibonacci: addi \$sp, \$sp, -12 sw \$ra, 8(\$sp) sw \$a0, 4(\$sp) slti \$t0, \$a0, 1 beq \$t0, \$zero, L1 li \$v0, 0 addi \$sp, \$sp, 12 jr \$ra L1: slti \$t0, \$a0, 2 beq \$t0, \$zero, L2 li \$v0, 1 addi \$sp, \$sp, 12 jr \$ra </pre>	- In vị trí - In prompt3 - In giá trị fibo - Kết thúc chương trình - Giảm \$sp để lưu \$ra trả về và giá trị n - $n \geq 1$ thì đến L1 - $n < 1$ thì : + Gán \$v0 = 0 + Tăng \$sp + Nhảy đến địa chỉ mà \$ra đang lưu trữ - $n \geq 2$ thì đến L2 - $n < 2$ thì: + Gán \$v0 = 0 + Tăng \$sp + Nhảy đến địa chỉ mà \$ra đang lưu trữ
--	---

L2: <pre> addi \$a0, \$a0, -1 jal fibonacci sw \$v0, 0(\$sp) lw \$a0, 4(\$sp) addi \$a0, \$a0, -2 jal fibonacci lw \$t0, 0(\$sp) add \$v0, \$v0, \$t0 lw \$a0, 4(\$sp) lw \$ra, 8(\$sp) addi \$sp, \$sp, 12 jr \$ra </pre>	- Giảm n đi 1 $\Leftrightarrow$ fibonacci(n – 1) - Nhảy lại hàm fibonacci - Lưu giá trị \$v0 vào stack - Lấy giá trị \$a0 ra khỏi stack - Giảm n đi 2 $\Leftrightarrow$ fibonacci(n – 2) - Nhảy lại hàm fibonacci - Lưu giá trị \$t0 vào stack - fibonacci(n) = fibonacci(n – 1) + fibonacci(n – 2) - Lấy giá trị \$a0 và \$ra ra khỏi stack - Tăng \$sp để lấy các giá trị tiếp theo
--	--

Chạy mô phỏng:

```

Enter a value: 4
Fibonacci (4) = 3

```

-----*Hết*-----